

Frontend Development Standards.

A unified guide for writing consistent, maintainable, and scalable frontend code. Built for clarity, designed for collaboration.

AUDIENCE

Engineering Team

DOCUMENT TYPE

Coding Standards

STATUS

Active

Table of Contents

01	Code Quality & Safety	03
02	File Structure & Organization	03
03	Naming Conventions	04
04	Styling Standards	05
05	Component Design Principles	06
06	React Best Practices	06
07	Accessibility (a11y)	07
08	Performance	07
09	Code Hygiene	07
10	Git & PR Practices	08
11	Documentation	08

01 Code Quality & Safety

► 1.1 Error Handling

- Every function and utility **must** be wrapped in a `try/catch` block where applicable — especially those involving async operations, API calls, or data transformations.
- Always log errors meaningfully and handle them gracefully without breaking the user experience.

► 1.2 Optional Chaining

- Use optional chaining (`?.`) and nullish coalescing (`??`) extensively to safeguard against `undefined` / `null` errors.
- Never assume nested object properties exist without explicit checks.

► 1.3 Type Safety

- Define explicit **types**, **interfaces**, and **enums** for every data structure, prop, function parameter, and return value.
- Avoid `any` — use `unknown` if the type is truly indeterminate and narrow it before use.
- Centralize shared types in a dedicated `types/` directory.

02 File Structure & Organization

► 2.1 File Size Limit

- A single `.tsx` file should **not exceed 200 lines** (including imports).
- If the file is growing beyond this limit, refactor by extracting reusable logic into child components, custom hooks, or utility functions.

► 2.2 Component Folder Structure

If a component has child components, it should reside in its **own folder** with the following structure:

```
ComponentName/  
├─ index.tsx           // Main component  
├─ components/         // Child components  
│   ├─ ChildA.tsx  
│   └─ ChildB.tsx  
├─ hooks/              // Component-specific hooks  
├─ utils/              // Component-specific utilities  
└─ types.ts            // Component-specific types
```

► 2.3 Imports

- Group imports logically: external libraries → internal modules → relative imports → styles.
- Use absolute imports (via `@/`) where possible for better readability.

03 Naming Conventions

Entity	Convention	Example
.tsx Component Files	PascalCase	<code>UserProfileCard.tsx</code>
Functions / Methods	camelCase	<code>fetchUserDetails()</code>
Utilities / Helpers	camelCase	<code>formatDate()</code>
Types / Interfaces	PascalCase	<code>UserProfile, IApiResponse</code>
Enums	PascalCase (UPPER_SNAKE members)	<code>UserRole.ADMIN</code>
Constants	UPPER_SNAKE_CASE	<code>MAX_RETRY_COUNT</code>
CSS Class Names	kebab-case	<code>user-card-wrapper</code>

Pro Tip: Consistent naming is the foundation of readable code. When in doubt, follow the patterns established in the existing codebase rather than introducing new conventions.

04 Styling Standards

► 4.1 Tailwind First

- Prioritize using **Tailwind utility classes** for font sizes, spacing, colors, layout, and responsiveness.
- Avoid writing custom CSS unless absolutely necessary.

► 4.2 Units

- When custom values are required, **always use** `rem`, never `px`.
- This ensures responsive scaling and consistency with the application's base font size.

► 4.3 Text Color Hierarchy

For non-branding text, default to **muted tones** using Tailwind's `zinc` or `gray` palette. Use only the following text color classes:

Class	Sample	Usage
<code>text-zinc-700</code>	 #3f3f46	Titles and primary headings
<code>text-zinc-600</code>	 #52525b	Body text, primary content
<code>text-zinc-500</code>	 #71717a	Descriptions, secondary text

Reserved: `text-zinc-400` is reserved exclusively for icons, muted/disabled text, and disabled button labels.

Do Not Use: `text-zinc-900` (or any 800+ shade) — it is too heavy for our visual style and breaks the muted, balanced look of the application.

- Avoid using arbitrary hex codes unless required by the brand guideline.

05 Component Design Principles

► 5.1 Single Responsibility

- Each component should do one thing and do it well.
- If a component handles multiple concerns, break it down.

► 5.2 Props Discipline

- Always define explicit `Props` interfaces for components.
- Avoid prop drilling beyond 2 levels — use composition or appropriate state management.
- Provide sensible defaults for optional props.

► 5.3 State Management

- Keep state as local as possible.
- Lift state up only when necessary for sharing.
- Use global state (Redux) only for genuinely shared application-level data.

06 React Best Practices

► 6.1 Hooks

- Custom hooks should be named with a `use` prefix (e.g., `useUserData`).
- Extract complex logic into custom hooks to keep components clean.

► 6.2 Memoization

- Use `useMemo` and `useCallback` thoughtfully — only when there's a clear performance benefit.
- Avoid premature optimization.

► 6.3 Keys in Lists

- Always provide stable, unique `key` props in lists. Never use array indices unless the list is static.

07 Accessibility (a11y)

- Use semantic HTML (`<button>` , `<nav>` , `<header>` , etc.) instead of generic `<div>` s where applicable.
- Always include `aria-label` or `aria-describedby` for interactive elements without visible text.
- Ensure color contrast meets at least WCAG AA standards.
- Make all interactive elements keyboard-accessible.

08 Performance

- Lazy-load routes and heavy components using `React.lazy`.
- Avoid unnecessary re-renders — profile with React DevTools when needed.
- Optimize images (use modern formats like WebP, lazy loading where appropriate).
- Debounce or throttle expensive operations triggered by user input.

09 Code Hygiene

- No commented-out code in commits — use version control history instead.
- No `console.log` statements in committed code (except intentional logging via a logger utility).
- Remove unused imports, variables, and dead code before raising a PR.
- Run linter and formatter (ESLint + Prettier) before every commit.

10 Git & PR Practices

► 10.1 Conventional Commits

Commit messages should follow the **conventional commits** format. Use the appropriate tag based on the nature of the change:

Tag	Meaning
<code>feat:</code>	A new feature or capability added to the codebase.
<code>fix:</code>	A bug fix that resolves an issue in existing functionality.
<code>refactor:</code>	Code changes that neither fix a bug nor add a feature (restructuring, renaming, cleanup).
<code>chore:</code>	Maintenance work like dependency upgrades, config changes, or housekeeping.
<code>style:</code>	Code style changes — formatting, whitespace, missing semicolons, etc. (no logic change).
<code>docs:</code>	Documentation-only changes such as updates to README, comments, or guides.
<code>perf:</code>	A code change that improves performance.
<code>test:</code>	Adding new tests or correcting existing ones.
<code>build:</code>	Changes to build system, bundlers, or external dependencies (webpack, npm, etc.).
<code>ci:</code>	Changes to CI configuration files and scripts (GitHub Actions, Jenkins, etc.).
<code>revert:</code>	Reverts a previous commit.

► 10.2 Pull Requests

- Keep PRs focused and reasonably sized — easier to review, easier to revert.
- Include a clear description, screenshots (for UI changes), and link relevant tickets.
- Self-review before requesting reviews from others.

11 Documentation

- Document complex logic with inline comments explaining *why* — not *what*.
- Maintain a README for each module/feature where non-trivial setup is required.
- Update relevant docs whenever changes affect public APIs, contracts, or developer workflows.

Personal Reminder: If you're someone who easily forgets what your own code does after a few weeks, make sure to add proper comments — especially for **functions, utilities, Redux slices, and custom hooks**. Use **JSDoc notation** and always include an example. Future-you (and the team) will thank you.

► 11.1 JSDoc Example

```
/**
 * Formats a date string into a human-readable format.
 *
 * @param date - ISO date string or Date object to format.
 * @param locale - Optional locale identifier (defaults to 'en-US').
 * @returns Formatted date string (e.g., "May 28, 2026").
 *
 * @example
 * formatDate("2026-05-28");           // "May 28, 2026"
 * formatDate(new Date(), "en-GB");    // "28 May 2026"
 */
export const formatDate = (date: string | Date, locale = 'en-US'): string => {
  // implementation...
};
```

Following this pattern makes your code self-documenting and dramatically reduces ramp-up time for anyone (including yourself) revisiting the code later.